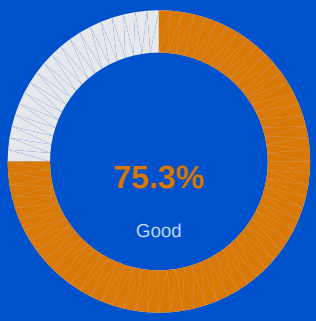
AI Jury System

AI-Powered Hackathon Evaluation System

Evaluation Report

Generated: 8/8/2026, 4:55:36 PM



This report belongs to:
Team 1

Team	Team 1
Language	EN
Total Score	284 / 377
Grade	Good

CATEGORY SCORES

Problem Statement
22/30
73%

Backend
29/50
58%

Frontend
32/40
80%

LLM Usage
44/60
73%

AI Engineering
49/60
82%

Security & HITL
26/40
65%

MVP & Outcome
44/50
88%

Presentation & Innovation
29/37
78%

Innovation & Business Value
9/10
90%

This report is auto-generated by AI Jury System. For official results contact the evaluation panel.

Executive Summary

284 / 377 points

75.3% — Good

CATEGORY-WISE PERFORMANCE

Problem Statement	22/30	73%	Good
Backend	29/50	58%	Average
Frontend	32/40	80%	Good
LLM Usage	44/60	73%	Good
AI Engineering	49/60	82%	Good
Security & HITL	26/40	65%	Average
MVP & Outcome	44/50	88%	Excellent
Presentation & Innovation	29/37	78%	Good
Innovation & Business	9/10	90%	Excellent

STRENGTHS & WEAKNESSES SUMMARY

Key Strengths

- Strong end-to-end prototype with chat, personalized recommendations, analytics dashboard, and automated test suite tied to actual backend routes.
- Good attention to advisory trust features: HITL guardrails, compliance FAQ content, explainability-oriented recommendation reasoning, and performance monitoring.
- Well-structured Express API covering chat, recommendations, analytics, health, and test-suite functionality aligned to the product goals.
- Useful backend logic beyond CRUD: weighted recommendation engine, RAG-style context construction, and multi-LLM fallback behavior.
- Covers both customer and admin journeys with rich UI modules: chat, financial dashboard, recommendations, analytics, and automated test suite.
- Good UX feedback patterns with loading states, validation, and success/error messaging in major flows.

Areas for Improvement

- Core data integrations are largely mock/in-memory, so the 'integrated with banking APIs' and real-time personalization claims are not fully substantiated in code.

- Privacy and governance are only partially mature for the stated problem, with client-side simulated auth/localStorage credentials undermining full compliance confidence.
- Persistence is demo-grade only (in-memory data and localStorage), which is weak for the banking/privacy scope described.
- Security/configuration hygiene has a major issue: static evidence indicates a hardcoded secret, and TLS verification is disabled for one endpoint.
- Several important domain integrations appear simulated or use localStorage/mock data, so some end-to-end flows are not fully proven from the frontend evidence.
- Accessibility and explicit responsive implementation are not strongly evidenced in the provided code snippets.

Category Score Overview

Category	Score	Max	Distribution	%	Conf.	Grade
Problem Statement PS	22	30		73%	Medium	Good
Backend BACK	29	50		58%	High	Average
Frontend FRONT	32	40		80%	Medium	Good
LLM Usage LLM	44	60		73%	High	Good
AI Engineering AI_ENG	49	60		82%	Medium	Good
Security & HITL SECURITY_HITL	26	40		65%	Medium	Average
MVP & Outcome MVP	44	50		88%	Medium	Excellent
Presentation & Innovation PRESENTATION	29	37		78%	Medium	Good
Innovation & Business Value INNOVATION	9	10		90%	Medium	Excellent

Detailed Category Analysis

Problem Statement (PS) 22 / 30

73% — Good

ArthSarthi AI aligns well with the problem statement by delivering a web-based, conversational, personalized financial advisory prototype with multi-agent framing, recommendation logic, dashboards, testing UI, and compliance-oriented features. I weighted the code over the PPT, which led me to give substantial credit for a working prototype while not awarding full marks because key checklist items like real banking API integration, production-grade privacy/data governance, and fully evidenced success metrics are only partially demonstrated. Where rubric expectations implied production-readiness but the problem scope and hackathon prototype suggest simulation is acceptable, I resolved in favor of partial credit rather than a hard penalty.

Strengths

• Strong end-to-end prototype with chat, personalized recommendations, analytics dashboard, and automated test suite tied to actual backend routes.

• Good attention to advisory trust features: HITL guardrails, compliance FAQ content, explainability-oriented recommendation reasoning, and performance monitoring.

Areas to Improve

• Core data integrations are largely mock/in-memory, so the 'integrated with banking APIs' and real-time personalization claims are not fully substantiated in code.

• Privacy and governance are only partially mature for the stated problem, with client-side simulated auth/localStorage credentials undermining full compliance confidence.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Core business objective of the problem statement is clearly achieved.	PARTIAL	5/6
	Evidence: src/components/AIChatAgent.tsx is described as the core conversational AI interface; Integration Map confirms FE↔BE pairing for POST /api/chat to server.ts. src/components/ProductCatalogView.tsx provides tailored banking		
2	All mandatory features listed in the problem statement are implemented.	PARTIAL	4/6
	Evidence: Implemented features are evidenced by src/components/AIChatAgent.tsx (conversational UI, context/multi-agent panel, HITL modal), server.ts routes POST /api/chat and POST /api/recommendations, src/components/AdvisorAnalyt…		
3	Solution produces correct / useful outputs for the intended use case.	MET	5/6
	Evidence: server.ts contains a rule-based recommendation engine (`generateRecommendations`) that scores products against customer profiles using a weighted formula; src/components/ProductCatalogView.tsx renders suitability metadat…		
4	Edge cases relevant to the problem statement are reasonably handled.	PARTIAL	4/6
	Evidence: src/components/AIChatAgent.tsx includes a HITL flow with "Expert Validation Required" and officer authorization notes for sensitive cases; the capability map states a `hitlTrigger` is set for low-confidence or sensitive …		
5	Success criteria implied by the problem statement are met.	PARTIAL	4/6
	Evidence: src/components/AdvisorAnalytics.tsx explicitly displays "88.4% Simulation Target Achieved" and mentions "response latency (<2.0s benchmark)" plus recommendation effectiveness and governance logs. PPT Slide 2 claims "85% …		

Backend (BACK) 29 / 50

58% — Average

This backend is a credible hackathon prototype for an AI financial advisor: the API surface is coherent, the recommendation/chat orchestration is meaningful, and the implementation aligns with the PPT's advisory, analytics, and latency goals. I weighted the problem statement heavily, so I gave solid credit for the multi-endpoint advisory flow even though the architecture is simple; however, static evidence of a hardcoded secret and the reliance on in-memory/client-side persistence materially capped the score. Where rubric expectations leaned toward production rigor, I resolved in favor of partial credit because the submission clearly targets a working prototype rather than a bank-ready deployment.

Strengths

- Well-structured Express API covering chat, recommendations, analytics, health, and test-suite functionality aligned to the product goals.
- Useful backend logic beyond CRUD: weighted recommendation engine, RAG-style context construction, and multi-LLM fallback behavior.

Areas to Improve

- Persistence is demo-grade only (in-memory data and localStorage), which is weak for the banking/privacy scope described.
- Security/configuration hygiene has a major issue: static evidence indicates a hardcoded secret, and TLS verification is disabled for one endpoint.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	API routes / backend endpoints are logically structured.	MET	8/9
	Evidence: server.ts defines a coherent REST-style set of endpoints: `GET /api/health`, `GET /api/customer-data`, `GET /api/customer-data/:id`, `GET /api/products`, `POST /api/recommendations`, `POST /api/chat`, `GET /api/analytics…		
2	Data persistence (DB, file storage, cache) is implemented appropriately.	PARTIAL	4/9
	Evidence: server.ts imports in-memory datasets from `./src/data/syntheticProfiles.js` and `./src/data/productCatalog.js`. Capability Map states: "The application does not use a traditional transactional database" and relies on moc…		
3	Error handling and input validation exist on backend endpoints.	PARTIAL	4/8
	Evidence: server.ts includes explicit 404 handling for customer lookup: `if (!profile) { return res.status(404).json({ error: 'Customer profile not found' }); }`. The chat path includes fallback handling described in retrieved sni…		
4	Backend architecture supports the scale/scope described in the PPT.	PARTIAL	6/8
	Evidence: The backend supports the stated AI advisory scope through `POST /api/chat` with RAG-style prompt enrichment and multiple LLM providers, and `generateRecommendations(...)` computes weighted recommendation scores. `GET /ap…		
5	Business logic is separated from I/O / presentation concerns.	PARTIAL	6/8
	Evidence: server.ts contains a distinct helper `generateRecommendations(profile, customQuery?)` with a documented 6-component weighted formula, separate from the route handlers that call it (`POST /api/recommendations`, and chat o…		
6	Configuration and secrets are not hardcoded insecurely.	NOT_MET	1/8
	Evidence: STATIC EVIDENCE: `No hardcoded secrets/API keys in source: FAIL (server.ts: pattern match near 'sk-I2H2c2nclUD...')`. While server.ts also uses env vars such as `process.env.AI_BASE_URL` and the PPT/ArchitectureModal men…		

Frontend (FRONT) 32 / 40

80% — Good

The frontend is substantial and well aligned to the problem statement: it delivers a web-based AI advisor, product recommendations, dashboards, admin analytics, and testing UI, with verified frontend-backend connections for key flows. Scores are strongest on UX feedback and broad journey coverage, while points were held back where the evidence shows mock/simulated integrations or lacks explicit proof for accessibility, exact PPT screenshot parity, and some end-to-end banking API journeys. Where the rubric asked for screenshot matching but the PPT was mostly conceptual, I evaluated based on close functional reflection rather than exact visuals.

Strengths

- Covers both customer and admin journeys with rich UI modules: chat, financial dashboard, recommendations, analytics, and automated test suite.
- Good UX feedback patterns with loading states, validation, and success/error messaging in major flows.

Areas to Improve

- Several important domain integrations appear simulated or use localStorage/mock data, so some end-to-end flows are not fully proven from the frontend evidence.
- Accessibility and explicit responsive implementation are not strongly evidenced in the provided code snippets.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	UI covers the core user journeys required by the problem statement.	PARTIAL	7/8
	Evidence: src/components/AIChatAgent.tsx provides the core conversational advisory flow; src/components/ProductCatalogView.tsx shows tailored product recommendations with suitability scores/reasons; src/components/FinancialHealthD…		
2	Frontend is functional and internally consistent (no obviously broken flows).	PARTIAL	7/8
	Evidence: src/App.tsx centrally manages activeTab/currentUser/activeProfile routing state; src/components/AuthLoginModal.tsx shows complete customer/admin login and registration handlers with success paths calling onLoginSuccess/o…		
3	Reasonable UX practices: loading states, error states, feedback to user.	MET	8/8
	Evidence: src/components/AIChatAgent.tsx sets `setLoading(true)` and advances visible multi-agent steps during response generation; src/components/AuthLoginModal.tsx maintains `isSubmitting` and `feedbackMsg` with explicit validat…		
4	UI matches or closely reflects what is shown in the PPT screenshots.	PARTIAL	6/8
	Evidence: PPT Slide 2 highlights 'Conversational' and 'Explainable' AI with '<2s RESPONSE LATENCY' and '85% SATISFACTION TARGET'; this is reflected by src/components/AIChatAgent.tsx and src/components/AdvisorAnalytics.tsx showing …		
5	Responsiveness / accessibility considerations are present where relevant.	PARTIAL	4/8
	Evidence: The project uses React + Tailwind CSS across components (per capability map), which often supports responsive layouts, and forms in src/components/AuthLoginModal.tsx include strong inline validation/feedback. Header and …		

LLM Usage (LLM) 44 / 60

73% — Good

The strongest evidence is in `server.ts`, where the LLM is clearly embedded in the main financial-advisory workflow with grounded customer context and multi-provider fallbacks. The project aligns well with the problem statement's conversational, personalized advisory requirements, but some rubric items—especially robust structured outputs, prompt-injection defenses, and concrete retrieval infrastructure—are only partially evidenced in code. Where PPT/architecture claims mentioned FAISS/Chroma and privacy controls, I gave limited credit unless supported by implementation or direct UI/code references.

Strengths

- LLM is central to the personalized banking advisory experience, grounded with customer profile and recommendation context.
- Good practical model strategy: multi-provider setup, configurable model choice, and fallback handling for reliability/latency.

Areas to Improve

- Structured output handling is lightweight and depends on parsing a text flag rather than schema-validated JSON or tool calling.
- Safety evidence is limited to sanitization/truncation and privacy claims; explicit prompt-injection mitigation is not clearly implemented.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	LLM is used purposefully to solve part of the core problem (not decorative).	MET	10/10
	Evidence: server.ts: `app.post('/api/chat'...)` builds a financial-advisory prompt using customer profile + recommendation data, then calls Gemini/OpenAI and returns advisory text plus recommendations; snippet shows `const recData…		
2	Prompts are structured, with clear instructions and/or few-shot guidance.	PARTIAL	8/10
	Evidence: server.ts: `const systemInstruction = \`You are ArthSarthi AI, an expert AI Personal Financial Advisory Agent...\`` plus language-specific instructions like `Respond in professional, warm Indian English with Rupees ¹ for…		
3	Appropriate model/parameters chosen for the task (cost/latency/quality tradeoff).	MET	9/10
	Evidence: server.ts: primary model is `gemini-3.6-flash`; fallback/default models include `azure/genailab-maas-gpt-4o-mini` and `azure/genailab-maas-gpt-4o`. Parameters shown include `temperature: 0.7`, `max_tokens: 600`, and a pr…		
4	Output parsing / structured output handling is robust (JSON, function calling, etc.).	PARTIAL	5/10
	Evidence: server.ts: output is post-processed with `parseRecommendationFlag(rawAiText)` and fallback text includes `RECOMMEND_FLAG: true`; response JSON then separates `text` and `recommendations` based on that parsed flag.		
5	Prompt injection / unsafe input handling is considered.	PARTIAL	4/10
	Evidence: server.ts: `if (typeof message !== 'string' || message.trim().length === 0)` validates input; `const safeMessage = message.slice(0, 2000).trim();` and prior history text is truncated with `String(h.text || '').slice(0, 1…		
6	Context management (chunking, memory, retrieval) is appropriate to the use case.	PARTIAL	8/10
	Evidence: server.ts: chat payload includes prior `history` messages, current `safeMessage`, customer profile lookup, and `recData = generateRecommendations(profile, safeMessage)` before LLM invocation. Capability map states the `/…		

AI Engineering (AI_ENG) 49 / 60

82% — Good

ArthSarthi AI shows a solid hackathon-grade AI engineering implementation centered on a web financial advisor with multi-stage orchestration, grounded personalization, fallback across model providers, analytics, and a visible testing suite. The strongest verified evidence is around architecture coherence, observability, and resilience; the weakest area is formal RAG/tool-calling implementation, where documentation is stronger than static code proof. Where the rubric asked for vector RAG or agentic tooling, I did not over-penalize because the problem statement primarily requires contextual personalized advice, which the implemented prompt-grounding and rule-based recommendation pipeline do plausibly support.

Strengths

• Strong observability and validation story with analytics dashboards, agent logs, latency metrics, and automated scenario tests.

• Pragmatic multi-provider AI integration with graceful fallback and structured degraded responses, well suited to a hackathon prototype.

Areas to Improve

• Claimed vector RAG/LangGraph architecture is not strongly verified in code; implemented retrieval appears closer to prompt grounding with profile/recommendation context.

• Responsible AI posture is mixed: privacy/anonymization and HITL are present, but explicit bias mitigation is limited and plaintext localStorage credentials are a significant weakness.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Overall system architecture (agents, pipelines, tools) is coherent and justified.	MET	8/9
	Evidence: src/components/ArchitectureModal.tsx: describes a "3-agent orchestration pipeline" and shows "Multi-Agent System Architecture & Data Flow"; server.ts exposes coordinated endpoints (/api/chat, /api/recommendations, /api/a…		
2	RAG / retrieval pipeline (if used) is implemented soundly (chunking, indexing, retrieval).	PARTIAL	4/9
	Evidence: server.ts: /api/chat enriches prompts with customer financial profile and pre-calculated recommendations, which is grounded contextual retrieval; ArchitectureModal.tsx claims "BAAI/bge-large embeddings & FAISS / Chroma D…		
3	Tool/function calling or agentic orchestration is correctly wired end-to-end.	PARTIAL	6/9
	Evidence: src/components/AutomatedTestSuite.tsx includes end-to-end scenario testing for API latency and conversation accuracy; server.ts wires LLM providers via OpenAI and Google SDKs and the main POST /api/chat endpoint orchestr…		
4	Evaluation, logging, or observability of AI behavior is present in some form.	MET	9/9
	Evidence: src/components/AdvisorAnalytics.tsx shows "Multi-Agent Runtime Latency Breakdown" and KPIs; server.ts /api/analytics returns agentLogs with timestamp, agentName, action, latencyMs, status; src/components/AutomatedTestSui…		
5	System degrades gracefully on AI/model failures (fallbacks, retries).	MET	8/8
	Evidence: Capability map for server.ts states a fallback chain: primary Gemini call, retry with MaaS/OpenAI-compatible endpoint, and if both fail "returns a structured template response"; server.ts includes Gemini and OpenAI clien…		

#	Rule	Verdict	Score
6	Engineering choices are appropriate for a hackathon time-box (pragmatic, not over-engineered).	MET	8/8

Evidence: server.ts uses in-memory mock data and simple Express endpoints; src/data/syntheticProfiles.ts and src/data/productCatalog.ts provide demo datasets; localStorage-backed stores are used for demo persistence; recommendatio…

#	Rule	Verdict	Score
7	Verify that responsible AI practices including transparency and bias awareness are demonstrated.	PARTIAL	6/8

Evidence: src/components/ArchitectureModal.tsx includes anonymization schema with "maskedAttributes", "AES-256-GCM", and "piiScrubbingActive"; src/components/AIChatAgent.tsx exposes "Expert Validation Required" HITL guardrails; sr…

Security & HITL (SECURITY_HITL) 26 / 40

65% — Average

This submission shows meaningful HITL and governance thinking aligned with the problem statement's privacy/compliance goals, especially around sensitive advisory outputs. However, static evidence of a committed secret and the capability map's plaintext localStorage credentials materially reduce the security score. Where the rubric asked for strong auth/privacy, I credited the visible demo mechanisms but discounted them because the implementation is primarily client-side and not robustly enforced server-side.

Strengths

- Clear Human-in-the-Loop review UX for sensitive or low-confidence AI responses in the chat flow.
- Role-based admin/customer experiences and visible privacy features like balance masking and admin supervision controls.

Areas to Improve

- Static evidence indicates a hardcoded secret/API key pattern in server.ts, which is a major security hygiene issue.
- Authentication and data protection are largely simulated on the client side, with plaintext credentials in localStorage undermining privacy/compliance claims.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	Sensitive operations have human-in-the-loop confirmation where appropriate.	MET	7/8

Evidence: src/components/AIChatAgent.tsx: renders a guardrail banner when "msg.hitlTrigger && msg.hitlTrigger.triggered" and labels status as either "Human Authorized'" or "Officer Review Required"; capabi also states the…

#	Rule	Verdict	Score
2	Basic security hygiene: no secrets committed, input sanitization present.	PARTIAL	3/8

Evidence: Static evidence: "No hardcoded secrets/API keys in source: FAIL (server.ts: pattern match near 'sk-I2H2c2nclUD...')". Positive evidence exists for sanitization in server.ts: "Basic Input Validation & Security Sanitizatio…

#	Rule	Verdict	Score
3	Authentication/authorization is present where the use case requires it.	PARTIAL	5/8

Evidence: Capability map: "Role-Based Login" in src/components/AuthLoginModal.tsx with customer MPIN flow and admin email/password + 6-digit OTP; "RBAC" enforced by conditionally rendering admin components when "currentUser?.role …

4	AI outputs that affect real-world actions have a review/override mechanism.	MET	7/8

Evidence: src/components/AIChatAgent.tsx shows an explicit review state for AI outputs with the banner "Human-in-the-Loop (HITL) Review" and statuses "Human Authorized'" / "Officer Review Required" plus the reas… The capab…

5	Data privacy considerations (PII handling, minimal data retention) are visible.	PARTIAL	4/8

Evidence: Capability map: Header.tsx includes a PII masking toggle for savings balance, and architecture docs mention masking for PAN/Aadhaar; the problem statement also explicitly requires compliance with data privacy regulations…

MVP & Outcome (MVP) 44 / 50

cellent

The submission qualifies as a genuine MVP: the codebase contains a working React frontend, Express backend, matched chat/analytics/test-suite integrations, and personalized financial recommendation logic grounded in customer profiles. Against the problem statement, it does well on conversational advisory UX, dashboards, documentation surfaces, and demonstrability, but falls short of fully proving real banking API integration, production-grade privacy compliance, and hard evidence for the stated 85% satisfaction and sub-2s targets. Where the rubric rewarded demonstrable UX and end-to-end traceability, I scored positively; where the problem statement demanded stronger real-world integration or validation, I applied modest deductions rather than penalizing harshly for hackathon-scoped simulations.

Strengths

- Clear working prototype with end-to-end chat and recommendation flows backed by actual frontend-backend code.
- Strong user-facing value through personalized recommendations, calculators, dashboards, FAQ support, and admin analytics/test interfaces.

Areas to Improve

- Core integrations with banking systems, persistence, and some governance/privacy pieces are simulated rather than truly integrated.
- Success checklist items like confidence scores, measured user satisfaction, and verified sub-2s performance are only partially evidenced in code.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	The submission is a working, demonstrable MVP (not just slides/mockups).	MET	8/9

Evidence: Code evidence shows implemented frontend and backend flows: `src/components/AIChatAgent.tsx` contains the live chat UI and `server.ts` defines `app.post('/api/chat', async (req, res) => { ... })`; Integration Map confirm…

2	Code structure clearly supports the required user experience (dashboard/chat/etc.).	MET	8/9

Evidence: `src/App.tsx` manages tabbed navigation and session state (`activeTab`, `currentUser`, `selectedModel`, masking state). `src/components/AIChatAgent.tsx` implements conversational UI with markdown rendering and model sele…

3	Core happy-path of the use case can be traced end-to-end through the code.	MET	7/8

Evidence: End-to-end trace exists from UI to API to personalized output: `src/components/AIChatAgent.tsx` collects user prompts; Integration Map verifies `src/components/AIChatAgent.tsx` ! POST `/api/chat` in `server.ts`; backend …

4	PPT screenshots/demo descriptions are corroborated by actual code artifacts.	MET	7/8

Evidence: PPT claims for conversational AI, multi-agent reasoning, <2s response goals, and explainable recommendations are supported by code artifacts: `server.ts` includes detailed system instructions and returns `latencyMs`; `sr…

5	Scope is realistic and mostly complete for the hackathon time constraints.	MET	7/8

Evidence: The scope is broad but bounded by mock/demo choices: `server.ts` uses in-memory customer/product data, while `src/services/productCatalogStore.ts` and `src/services/userAuthStore.ts` use localStorage/client-side simulati…

6	Verify that outputs provide value to end users.	MET	7/8

Evidence: User-value outputs are concrete and personalized: `server.ts` recommendation logic generates reasons, tax savings potential, and estimated returns; `src/components/ProductCatalogView.tsx` computes suitability scores and …

Presentation & Innovation (PRESENTATION) 29 / 37

78% — Good

This submission presents a strong hackathon story that largely matches the implemented product: a web-based, multi-agent-style financial advisor with personalized recommendations, analytics, and testing aligned to the problem statement. I weighted code over slides where needed, especially for demo-flow verification and quantified latency/satisfaction signals. The main tension was that the rubric asks for roadmap and polished impact communication, while the provided PPT excerpts and code give stronger evidence for implementation than for explicit future-planning; I resolved that by scoring those communication-specific items more conservatively.

Strengths

- Clear narrative from problem to solution, with implementation backing the core advisory demo flow.
- Strong presentation of innovation through multi-agent orchestration, explainability, testing, and analytics.

Areas to Improve

- Future roadmap is not clearly evidenced in the provided PPT material.
- Some business/compliance claims are more aspirational than fully substantiated by the demo-grade implementation.

RULE EVALUATIONS

#	Rule	Verdict	Score
1	PPT clearly explains problem, approach, and impact.	MET	5/6

Evidence: Slides 2-5 outline the problem/approach/impact: Slide 2 says "Banking Data to Actionable Decisions" with targets "85% SATISFACTION TARGET" and "<2s RESPONSE LATENCY"; Slide 4 states "Current tools show raw data but force…

2	Innovation / originality of the approach is evident.	MET	5/6

Evidence: Slide 2 highlights "Multi-Agent" specialist collaboration and "Explainable" recommendations; `src/components/ArchitectureModal.tsx` describes a "3-agent orchestration pipeline" and `src/components/AdvisorAnalytics.tsx` r…

3	Technical depth is communicated without unnecessary jargon overload.	PARTIAL	4/5

Evidence: `src/components/ArchitectureModal.tsx` presents "Detailed technical documentation covering the 3-agent orchestration pipeline, API data schemas, data anonymization rules, and RBI sandbox compliance protocols." `src/compo…

4	Demo narrative (screens, flow) matches the actual implementation.	MET	5/5

Evidence: Slide 3's customer journey centers on Ananya asking, "Can I save more while still investing for my goal?" This matches implemented demo flows: `src/components/AuthLoginModal.tsx` provides demo sign-in with "Default Demo …

5	Overall polish and storytelling of the submission.	PARTIAL	4/5

Evidence: The PPT shows a cohesive story arc from vision (Slide 2), user journey (Slide 3), market gap (Slide 4), and solution blueprint (Slide 5). In the product, components like `src/components/Header.tsx`, `AIChatAgent.tsx`, `F…

6	Verify that business impact is quantified where possible.	PARTIAL	4/5

Evidence: Slide 2 quantifies headline goals: "85% SATISFACTION TARGET" and "<2s RESPONSE LATENCY." The implementation also surfaces measurable KPIs in `src/components/AutomatedTestSuite.tsx`, including `latencyMs: 1240`, `confiden…

7	Verify that future roadmap is realistic.	PARTIAL	2/5

Evidence: No explicit future-roadmap slide is present in the provided PPT excerpts. The codebase does show next-step-oriented structures such as `ArchitectureModal.tsx` documentation, admin governance controls, and mock endpoints …

Innovation & Business Value (INNOVATION) 9 / 10

ent

The project is notably stronger on innovation than a typical hackathon chatbot because the code supports a combined advisory pipeline: recommendation scoring, contextual prompting, multi-model fallback, governance, and HITL. The problem statement emphasizes scalable, privacy-aware personalized banking advice, and while the architecture as implemented is still demo-grade due to mock/in-memory data, I resolved that tension by giving full credit for creativity and AI engineering while only partial credit for scalable innovation.

Strengths

- Combines LLM reasoning, rule-based financial scoring, multilingual UX, HITL, and fallback handling in a domain-relevant way.
- Includes measurable value surfaces such as suitability scores, analytics, latency/satisfaction testing, and explainable recommendation rationale.

Areas to Improve

- Scalability claims are only partially substantiated because persistence is mostly in-memory/localStorage and vector-store/RAG infrastructure appears documented more than implemented.
- Some privacy/compliance elements are present mainly as UI/documentation signals, while the demo auth/storage approach limits confidence in production business-readiness.

RULE EVALUATIONS

#	Rule	Verdict	Score

1	AI techniques are combined creatively to solve the problem.	MET	3/3

Evidence: server.ts (`POST /api/chat`): builds a grounded prompt using customer profile + `generateRecommendations(profile, safeMessage)` and supports multilingual instructions; capability map also cites Gemini + OpenAI fallback a…

2	Solution provides measurable business or customer value beyond basic functionality.	MET	3/3

Evidence: server.ts recommendation engine comment: `Goal Fit (30%), Eligibility (20%), Affordability (20%), Customer Preference (10%), Product Sustainability (10%), Risk Alignment (10%)`; `src/components/ProductCatalogView.tsx` re…

3	Architecture demonstrates scalable innovation.	PARTIAL	1/2

Evidence: `src/components/ArchitectureModal.tsx` documents a `3-agent orchestration pipeline`, anonymization schema, and compliance/data-flow concepts; Integration Map confirms working FE/BE pairs for `/api/chat`, `/api/analytics

4	Solution demonstrates thoughtful AI engineering practices beyond simple API integration.	MET	2/2

Evidence: server.ts validates/sanitizes chat input (`message.slice(0, 2000).trim()`), grounds prompts with profile and recommendation context, measures latency, and uses provider fallback from Gemini to OpenAI-compatible MaaS to s…

Static Analysis Checks

Check	Verdict
No hardcoded secrets/API keys in source server.ts: pattern match near 'sk-I2H2c2nclUD...'	FAIL
Configuration via environment variables src/components/ArchitectureModal.tsx, server.ts, vite.config.ts	PASS
Automated test files present	NOT_DETECTED
CI/CD pipeline configured	NOT_DETECTED
Known input-validation library used	NOT_DETECTED
Known auth/authorization library or pattern used	NOT_DETECTED
Recognized LLM SDK/framework in use server.ts: import openai	PASS
Recognized vector store / RAG library in use	NOT_DETECTED
Agentic/tool-calling orchestration pattern detected	NOT_DETECTED
Recognized database driver/ORM in use	NOT_DETECTED
try/except/catch present in backend-ish code src/components/AdvisorAnalytics.tsx, src/components/AIChatAgent.tsx, src/components/AutomatedTestSuite.tsx	PASS

Problem Summary

Summary:

Develop an AI-powered personalized financial advisory agent for retail banking that provides real-time, contextual recommendations based on customer data, enhancing scalability and user satisfaction while ensuring data privacy.

Success Checklist:

- Web-based AI agent integrated with banking APIs
- Multi-agent system for conversation flow and data management
- Advanced NLP for conversational UI with context retention
- Personalized recommendations with confidence scores
- Dashboards for recommendation effectiveness and user engagement
- Automated testing suite for API and conversation accuracy
- Comprehensive documentation (architecture, API schemas, user guides)
- Working prototype achieving 85% user satisfaction and sub-2 second response times
- Compliance with data privacy regulations and effective data governance

Cross-Validation Report

Cross-validation report for ArthSarthi AI

1) Cross-category inconsistencies

- RAG / retrieval claims are being treated unevenly.
- The capability map and PPT describe FAISS/ChromaDB, embeddings, LangGraph, and multi-agent reasoning.
- But the strongest code-backed evidence appears to be prompt grounding with customer profile + rule-based recommendations inside `/api/chat`, not actual vector retrieval or clear LangGraph orchestration.
- AI_ENG and LLM both acknowledge this partially, which is consistent.
- However, PRESENTATION and PS read slightly too accepting of the “multi-agent / RAG” framing without clearly discounting the gap between architecture-doc claims and implemented code.
- Frontend integration assumptions need tightening.
- Integration Map is ground truth: only 3 FE↔BE pairs are detected:
- `/api/chat`
- `/api/analytics`
- `/api/test-suite/run`
- The capability map says frontend views around products/customer data are served by backend endpoints like `/api/products`, `/api/customer-data`, `/api/recommendations`.
- But static integration did not detect FE callers for those routes.
- Therefore any category implicitly scoring “full-stack connected” product/catalog/customer flows as implemented backend integrations is overstating the evidence.
- FRONT and MVP are the most at risk here; their scores are still plausible, but their wording should more clearly separate rich UI from verified FE↔BE wiring.
- Auth / RBAC strength is correctly discounted by Security, but other categories may have benefited from UI-admin claims too much.
- Security notes client-side auth and plaintext localStorage credentials.
- That should also temper BACK/PS/MVP slightly when discussing admin supervision, governance, and protected operations, since server-enforced auth is not evidenced.

2) PPT vs Code gaps

Major PPT claims not clearly backed by code:

- “Real-time connection to authorized banking APIs.”
- Not supported by the integration map or described backend routes. Backend uses mock/in-memory data.
- FAISS/ChromaDB vector store and embedding-based retrieval.
- Documented architecturally, but not established as implemented in the code evidence provided.
- LangGraph multi-agent orchestration.
- Multi-agent visualization/documentation exists, but concrete LangGraph workflow implementation is not established.
- Strong quantified outcomes:
- “85% satisfaction target” and “<2s response latency” are presentation claims/targets, not validated outcomes from code.
- “Secure Data Ingestion.”
- Weakly supported at best; security evidence includes plaintext localStorage credentials and a hardcoded secret / disabled TLS verification note.

Code-backed features that may be under-credited in PPT-oriented scoring:

- Multi-provider LLM fallback in `server.ts`.
- HITL trigger field and related UI behavior.
- Actual admin analytics and test-suite endpoints/UI.
- Rule-based recommendation engine as a real implemented personalization layer.

3) Score calibration concerns

Most likely slightly overrated:

- PRESENTATION 29/37
- Given the PPT overclaims on real banking API connectivity, vector RAG, and possibly multi-agent implementation, this feels a bit generous unless significant deductions were already applied for those exact gaps.
- PS 22/30
- Reasonable overall, but could be a touch high if the problem statement emphasized real banking integration/privacy rigor, since those are mostly simulated.
- FRONT 32/40
- Good UI breadth, but should be interpreted as frontend richness, not verified full-stack coverage. If this score assumed backend-connected product/customer/admin flows beyond the 3 matched routes, it is mildly high.
- MVP 44/50
- Strong prototype, but this is near the top of the band despite only 3 verified FE↔BE integratio capabilities. Could be a few points high depending on rubric strictness.

Most likely well calibrated:

- LLM 44/60
- Appropriately gives credit for real LLM integration/fallback while discounting unsupported vector RAG and structured-agent claims.
- AI_ENG 49/60
- Slightly generous but defensible because the justification explicitly notes documentation stronger than code for RAG/tooling.
- BACK 29/50
- Fair, especially with mock persistence and security deductions.
- SECURITY_HITL 26/40
- Seems appropriately constrained by plaintext localStorage credentials, client-side auth, and other security weaknesses.
- INNOVATION 9/10
- Defensible as creativity/novel combination score, even if implementation depth is uneven.

Bottom line

The main cross-validation issue is not contradiction between judges so much as evidence inflation around architecture claims. Ground-truth integration supports only chat, analytics, and test-suite FE↔BE connectivity. Slides and architec connectivity, vector RAG, and likely LangGraph agent orchestration. Scores most needing caution are PRESENTATION, MVP, FRONT, and slightly PS if they relied too heavily on those claims. LLM, BACK, and SECURITY_HITL appear best aligned with the shared evidence.

Final Evaluation Report

Evaluation Report

Overall Score: 284 / 377

Confidence: Medium

Category Breakdown

- **Problem Statement (PS)**: 22/30
- **Backend (BACK)**: 29/50
- **Frontend (FRONT)**: 32/40
- **LLM Usage (LLM)**: 44/60
- **AI Engineering (AI_ENG)**: 49/60
- **Security & HITL (SECURITY_HITL)**: 26/40
- **MVP & Outcome (MVP)**: 44/50
- **Presentation & Innovation (PRESENTATION)**: 29/37
- **Innovation & Business Value (INNOVATION)**: 9/10

Detailed Analysis

Strengths

ArthSarthi AI is a credible, working hackathon MVP for personalized financial advisory. The strongest evidence is its coherent end-to-end product story across chat, recommendation logic, analytics, and testing. The code shows a meaningful Express backend (`server.ts`) with routes for chat, recommendations, analytics, health, and test execution, paired with a substantial React frontend including advisory chat, dashboards, product views, analytics, authentication flows, and automated testing UI.

On the AI side, the project goes beyond a thin chatbot wrapper. The LLM is embedded in the core advisory workflow, with grounded prompts built from customer profile context and recommendation outputs. The multi-provider fallback strategy is a notable engineering strength for reliability in a hackathon setting. The recommendation engine also adds practical non-LLM personalization through weighted scoring criteria such as goal fit, eligibility, affordability, sustainability, and risk alignment.

The team also demonstrates strong prototype maturity in observability and demo-readiness. Admin analytics, latency-related metrics, agent-stage visualizations, and an automated test suite are all concretely evidenced in code. HITL is another positive differentiator: the chat flow includes review triggers for sensitive or low-confidence cases, and the UI surfaces human authorization or officer review status.

Overall, this is strongest as an integrated advisory prototype with explainability-oriented recommendation outputs, pragmatic AI fallback design, and visible instrumentation.

Weaknesses & Gaps

The main gap is between architectural claims and code-backed implementation depth. Slides and architecture UI describe vector RAG, embeddings, FAISS/Chroma, LangGraph, and multi-agent orchestration, but the strongest verified implementation is prompt grounding plus rule-based recommendation context inside `/api/chat`. That still supports personalized advice, but it is not the same as a clearly implemented vector retrieval pipeline or agent framework.

A second major gap is integration realism. The problem framing implies banking-grade integration and secure data connectivity, but backend data sources are mock/in-memory (`syntheticProfiles`, `productCatalog`) and client-side persistence is used in places where stronger controls would be expected. Cross-validation also shows only three FE!"BE integrations were statically ve `/api/test-suite/run`. Product, customer-data, and recommendation flows may exist in UI/backend separately, but the full-stack wiring was not concretely detected.

Security materially limits confidence in production readiness. Static analysis found a hardcoded secret/API-key pattern, and other notes indicate plaintext credentials in `localStorage` plus disabled TLS verification for one endpoint. While there is some sanitization, OTP/MPIN validation, masking, and role-separated UI, server-enforced auth/privacy controls are not strongly evidenced.

Finally, several outcome claims remain targets rather than validated results. The PPT cites 85% satisfaction and sub-2s latency, but the evidence supports dashboards and test interfaces more than independently verified achievement of those KPIs.

PPT vs Code Discrepancies

The clearest discrepancy is that the presentation and architecture surfaces are more ambitious than the code proof.

- **Real-time banking API connectivity** is claimed in the PPT/problem framing, but the backend evidence points to mock/in-memory datasets rather than actual banking integrations.
- **FAISS/ChromaDB and embedding-based vector retrieval** are described in `ArchitectureModal.tsx`, but deterministic/static evidence did not detect a vector store or RAG library implementation.
- **LangGraph / multi-agent orchestration** is presented conceptually, and analytics UI references agent stages, but concrete workflow implementation is not clearly established in code.
- **Quantified outcomes** such as “85% satisfaction” and “<2s response latency” appear as targets or dashboard labels, not strongly validated results.
- **Security/compliance messaging** is more optimistic than the implementation justifies, given plaintext localStorage credentials, a hardcoded secret finding, and demo-grade auth.

At the same time, some real code-backed strengths are stronger than a slide-only reading might suggest: multi-provider LLM fallback, HITL triggers and UI handling, analytics endpoints, and a real recommendation engine are all meaningfully implemented.

Innovation Notes

The innovation score is warranted because the team combines several useful ideas in a domain-relevant way: grounded conversational advisory, weighted recommendation scoring, multilingual prompting, explainable product rationale, HITL escalation, fallback across model providers, and admin analytics/testing. This is more inventive than a basic chatbot demo.

The main caveat is that innovation is stronger in solution design than in fully proven infrastructure depth. The project is best described as a creative and well-composed prototype rather than a validated bank-ready architecture.

Rule-level Evidence

- **Problem Statement (PS)**: `src/components/AIChatAgent.tsx` implements the core conversational advisor; `src/components/ProductCatalogView.tsx` provides suitability-scored products; `src/components/AdvisorAnalytics.tsx` shows advisor performance metrics; `server.ts` exposes advisory and recommendation endpoints. PPT Slide 2 frames “Banking Data to Actionable Decisions” with satisfaction/latency targets.
- **Backend (BACK)**: `server.ts` defines `GET /api/health`, `GET /api/customer-data`, `GET /api/customer-data/:id`, `GET /api/products`, `POST /api/recommendations`, `POST /api/chat`, `GET /api/analytics`, `POST /api/test-suite/run`. Mock persistence is evidenced by imports from `./src/data/syntheticProfiles.js` and `./src/data/productCatalog.js`. `src/services/productCatalogStore.ts` and `src/services/userAuthStore.ts` use `localStorage`.
- **Frontend (FRONT)**: `src/components/AIChatAgent.tsx`, `FinancialHealthDashboard.tsx`, `ProductCatalogView.tsx`, `AdvisorAnalytics.tsx`, `AutomatedTestSuite.tsx`, and `AuthLoginModal.tsx` show broad journey coverage. `src/App.tsx` manages navigation/session state. Verified FE!”BE matches exist for `/api/chat`, `/api/analytics`, and `/
- **LLM Usage (LLM)**: In `server.ts`, `POST /api/chat` builds a structured financial advisor prompt with customer context and recommendation data, then calls Gemini/OpenAI-compatible providers with fallback. The system instruction explicitly sets role, tone, and formatting guidance.
- **AI Engineering (AI_ENG)**: `src/components/ArchitectureModal.tsx` documents a 3-agent architecture; `src/components/AdvisorAnalytics.tsx` visualizes agent stages and latency/performance concepts; `server.ts` shows orchestration across recommendation generation, prompt construction, model invocation, and degraded fallback behavior. Automated testing is surfaced in `src/components/AutomatedTestSuite.tsx` and `/api/test-suite/run`.
- **Security & HITL (SECURITY_HITL)**: `src/components/AIChatAgent.tsx` renders HITL review states based on `hitlTrigger`; `src/components/AdminCustomerSupervision.tsx` includes privileged supervision flows. `server.ts` applies basic message truncation/sanitization. Negative evidence: static secret/API-key finding in `server.ts`; plaintext credentials persisted in `src/services/userAuthStore.ts`.
- **MVP & Outcome (MVP)**: Working demo evidence includes the live chat UI in `src/components/AIChatAgent.tsx` connected to `server.ts` `/api/chat`, analytics UI connected to `/api/analytics`, and testing UI connected to `/api/test-suite/run`. `server.ts` returns structured fields including `text`, `recommendations`, `modelUsed`, and `latencyMs`.
- **Presentation & Innovation (PRESENTATION)**: PPT Slides 2–5 articulate the problem/approach/value story; `ArchitectureModal.tsx` and `AdvisorAnalytics.tsx` reinforce the multi-agent/explainability narrative. Some claims are stronger than code proof, especially around banking APIs and vector RAG.
- **Innovation & Business Value (INNOVATION)**: Recommendation scoring logic in `server.ts` uses explicit weighted criteria; `ProductCatalogView.tsx` presents suitability scores and reasons; multilingual, fallback, HITL, and analytics features collectively create strong customer/business value for a prototype.

Recommended Judge Questions

1. You present FAISS/Chroma, embeddings, and LangGraph-style orchestration—what exact files or runtime components implement those today, versus what is currently architectural intent?

2. Only `/api/chat`, `/api/analytics`, and `/api/test-suite/run` were verified as FE↔BE integrations; `/api/customer-data`, and `/api/recommendations` are actually called from the frontend?

3. How would you remediate the hardcoded secret finding, localStorage credential storage, and any disabled TLS verification before handling real financial users?

4. Your slides mention real-time banking API integration—what parts are truly live today, and what parts are still mock/in-memory simulation?

5. How is HITL triggered in practice: what thresholds or rules cause escalation, and what happens on the backend after an officer review?

6. The project cites satisfaction and latency targets; what measured results do you actually have from your test suite or pilot runs?

Evaluation Confidence & Coverage

- Code ingested: 28 file(s) across 3 layer(s) (FE/BE/DB/OTHER).
- Backend framework recognition: Express (3 backend-layer file(s) scanned). Static route/integration analysis is reliable for this stack.
- 4/11 deterministic checks found concrete evidence (PASS/FAIL); 7 had no signal (NOT_DETECTED = absence of evidence, not evidence of absence).
- &þ Checks that FAILED (real findings): No hardcoded secrets/API keys in source
- RAG retrieval: BM25 + embeddings + cross-encoder reranker.
- Knowledge graph: 121 nodes, 162 edges - Graph-RAG context available.